Project Koios Optimization
User Manual

Project Koios

Contents

1 Alignment **1**
- 1.1 Installation . . . 1
- 1.2 Mathematical conventions . . . 1
- 1.3 Choosing an interface . . . 1
- 1.4 Ownership boundary . . . 2

2 Scalar complex-unitary Procrustes **3**
- 2.1 Problem and orientation . . . 3
- 2.2 Why the SVD solves the problem . . . 3
- 2.3 Exact procedure . . . 4
- 2.4 Numerical controls . . . 4
- 2.5 Numerical rank . . . 5
- 2.6 Optimal residual and roundoff evidence . . . 5
- 2.7 Backend validation bounds . . . 6
- 2.8 Using the scalar API . . . 6
- 2.9 Result interpretation . . . 7
- 2.10 Failure handling . . . 7

3 Batched complex-unitary Procrustes **8**
- 3.1 Batched problem . . . 8
- 3.2 Per-item mathematical procedure . . . 8
- 3.3 Array evidence and shapes . . . 9
- 3.4 Fail-closed use of transformations . . . 9
- 3.5 Empty batches . . . 10
- 3.6 Whole-batch failure policy . . . 10
- 3.7 Memory model . . . 10

4 NumPy batched Procrustes **11**
- 4.1 Input contract . . . 11
- 4.2 Normal construction: defensive copies . . . 11
- 4.3 Solving and applying the transformation . . . 12
- 4.4 Complete worked example . . . 12
- 4.5 Zero-copy immutable construction . . . 13
- 4.6 Result construction and persistence . . . 14
- 4.7 Backend validation performed before return . . . 15
- 4.8 Errors and recovery . . . 15

5 SciPy batched Procrustes status **16**

References **17**

Chapter 1

Alignment

Alignment selects a mathematical transformation between already prepared numerical representations. Optimization owns the transformation-selection problem, numerical controls, and reconstructable evidence. It does not own the physical or scientific meaning of the matrices.

1.1 Installation

The dependency-free scalar contract is installed with the base distribution. The NumPy backend requires the NumPy extra:

```
python -m pip install "projectkoios-optimization[numpy]"
```

The distribution requires Python 3.14 or newer. NumPy support requires NumPy 2.3 or newer and earlier than NumPy 3.

1.2 Mathematical conventions

For a complex matrix A:

- $A^\dagger$ is the conjugate transpose;
- $\|A\|_F^2 = \sum_{ij} |A_{ij}|^2$ is the squared Frobenius norm;
- $\|A\|_{\max} = \max_{ij} |A_{ij}|$ is the maximum-entry norm; and
- I_n is the $n \times n$ identity matrix.

All Procrustes transformations in this manual act on the *right*. If A is the source and Q is the returned transformation, the aligned matrix is AQ, never QA. The library does not transpose, conjugate, reorder, or reinterpret consumer axes except when explicitly forming $A^\dagger B$.

1.3 Choosing an interface

Use scalar Procrustes for one pair of dependency-free tuple matrices. Use NumPy batched Procrustes for an array with one or more leading batch dimensions. The scalar and batched families share the same objective, rank cutoff, residual reconstruction, and validation tolerances.

The scalar contract withholds a transformation when numerical rank is deficient. The batched contract retains one SVD-selected completion per item for homogeneous array output, but marks

deficient completions as diagnostic and provides a fail-closed accessor for transformations that are unique for every item.

1.4 Ownership boundary

Before calling this library, a consumer must decide what each row, column, and leading batch index means and must convert physical quantities into suitable unit-free numerical representations. PhysKit may own that adaptation. After the solve, KSD or another consumer may interpret rank and residual evidence under separately declared scientific policy.

The following are not supplied by Alignment:

- physical units or operator semantics;
- correspondence, topology, path ordering, or periodic closure;
- a scientific rank threshold or acceptable residual;
- a scientific disposition or human acceptance decision; and
- a claim that a diagnostic deficient completion is unique.

Full numerical rank and a small software residual are evidence, not scientific validation.

Chapter 2

Scalar complex-unitary Procrustes

2.1 Problem and orientation

Let $A, B \in \mathbb{C}^{m\times n}$ be finite, nonempty matrices with identical shape. The scalar capability solves the complex-unitary form of the orthogonal Procrustes problem [1, 2]:

$$\min_{Q\in\mathbb{C}^{n\times n}} \|AQ - B\|_F \quad \text{subject to} \quad Q^\dagger Q = I_n.$$

The transformation is square even when $m \neq n$, because it acts on the n columns of A. Define the cross matrix

$$C = A^\dagger B \in \mathbb{C}^{n\times n}$$

and a complete singular-value decomposition

$$C = U\Sigma V^\dagger, \qquad \Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_n), \qquad \sigma_1 \geq \cdots \geq \sigma_n \geq 0.$$

When C has full numerical rank, the selected right-unitary polar factor is

$$Q = UV^\dagger,$$

and the consumer applies it as $A_{\text{aligned}} = AQ$.

2.2 Why the SVD solves the problem

For any unitary Q, expand the squared objective:

$$\begin{aligned}\|AQ - B\|_F^2 &= \operatorname{tr}\Big((AQ - B)^\dagger (AQ - B)\Big) \\ &= \|A\|_F^2 + \|B\|_F^2 - 2\operatorname{Re}\operatorname{tr}\Big(Q^\dagger A^\dagger B\Big).\end{aligned}$$

The two norm terms do not depend on Q. Minimization is therefore equivalent to maximizing $\operatorname{Re}\operatorname{tr}(Q^\dagger C)$. Substituting $C = U\Sigma V^\dagger$ and $Q = UV^\dagger$ gives

$$\operatorname{Re}\operatorname{tr}(Q^\dagger C) = \operatorname{tr}(\Sigma) = \sum_{j=1}^{n} \sigma_j.$$

The von Neumann trace inequality bounds every unitary choice by this nuclear norm [5], so the selected Q attains the global minimum. This SVD construction is also the unitary polar factor of

C [3]. If C is nonsingular, that factor is unique. If C is singular, directions in its null spaces are unconstrained by the objective and multiple unitary completions attain the same minimum. The software rank policy decides whether binary64 evidence is treated as nonsingular for the public contract; it does not claim exact algebraic rank. Standard SVD and numerical-rank background is given by Golub and Van Loan [4].

2.3 Exact procedure

For one definition, the scalar solver performs the following procedure.

1. Validate exact immutable tuple containers, exact built-in complex entries, finiteness, equal shape, and the exact tolerance-record type.
2. Form $C = A^\dagger B$ with checked complex binary64 products and deterministic componentwise summation.
3. Request a complete square SVD $C = U\Sigma V^\dagger$ from the selected backend.
4. Validate the complete spectrum, factor unitarity, and reconstruction of C.
5. Calculate the additive numerical-rank cutoff and numerical rank.
6. Calculate source power, target power, the optimal residual, and its roundoff cutoff.
7. At full numerical rank, form $Q = UV^\dagger$, validate right unitarity, and compare its direct residual with the singular-value optimum.
8. Return a relationally validated immutable result with backend identity.

No full transformation is formed or returned on the deficient scalar branch.

2.4 Numerical controls

The tolerance record contains four exact built-in floats:

$$\tau^{\text{abs}}_{\text{rank}} \geq 0, \qquad 0 \leq \tau^{\text{rel}}_{\text{rank}} < 1,$$
$$\tau^{\text{abs}}_{\text{validation}} \geq 0, \qquad 0 \leq \tau^{\text{rel}}_{\text{validation}} < 1.$$

Neither the rank pair nor the validation pair may be identically zero. The defaults are

$$\tau^{\text{abs}}_{\text{rank}} = 0, \qquad \tau^{\text{rel}}_{\text{rank}} = 10^{-12},$$
$$\tau^{\text{abs}}_{\text{validation}} = 10^{-12}, \qquad \tau^{\text{rel}}_{\text{validation}} = 10^{-10}.$$

Construct an explicit policy when the defaults are not appropriate:

```
from projectkoios.optimization.alignment.procrustes import (
    ComplexUnitaryProcrustesTolerance,
)

tolerance = ComplexUnitaryProcrustesTolerance(
    rank_absolute=0.0,
    rank_relative=1.0e-12,
```

```
    validation_absolute=1.0e-12,
    validation_relative=1.0e-10,
)
```

These are software controls. They are not scientific overlap thresholds.

2.5 Numerical rank

For largest singular value σ_1, the applied cutoff is

$$\tau_{\text{rank}} = \tau_{\text{rank}}^{\text{abs}} + \tau_{\text{rank}}^{\text{rel}}\sigma_1.$$

The reported numerical rank is

$$r = \sum_{j=1}^{n} \mathbf{1}[\sigma_j > \tau_{\text{rank}}].$$

The comparison is deliberately strict: $\sigma_j = \tau_{\text{rank}}$ is deficient. If $r = n$, status is `UNIQUE` and Q is present. If $r < n$, status is `RANK_DEFICIENT` and `transformation` is `None`.

Rank-deficient SVDs contain arbitrary bases for null spaces. Consequently, a complete backend SVD can produce a unitary completion, but the scalar API does not expose that completion as an identified transformation.

2.6 Optimal residual and roundoff evidence

Let

$$p_A = \|A\|_F^2, \qquad p_B = \|B\|_F^2.$$

The mathematical optimum is

$$r_*^2 = p_A + p_B - 2\sum_{j=1}^{n} \sigma_j(C).$$

The implementation evaluates this cancellation-prone identity after scaling, following the general principle that numerical error must be bounded relative to the magnitudes of the accumulated terms [6]. Define

$$s = \max(p_A, p_B, 2\sigma_1, \ldots, 2\sigma_n)\,.$$

If $s = 0$, both the residual and its cutoff are zero. Otherwise, define

$$\begin{aligned}
\widehat{r} &= \frac{p_A}{s} + \frac{p_B}{s} - \sum_{j=1}^{n} \frac{2\sigma_j}{s}, \\
\gamma_n &= \max(64, 8(n+2))\,\epsilon_{64}, \\
\widehat{\delta} &= \gamma_n \left(\frac{p_A}{s} + \frac{p_B}{s} + \sum_{j=1}^{n} \frac{2\sigma_j}{s} \right), \\
\delta &= s\widehat{\delta}.
\end{aligned}$$

Here ϵ_{64} is binary64 machine epsilon. If $|\widehat{r}| \le \widehat{\delta}$, the represented squared residual is exactly zero. If $\widehat{r} < -\widehat{\delta}$, reconstruction fails with `FloatingPointError`. Otherwise the reported value is $r_*^2 = s\widehat{r}$.

The result retains p_A, p_B, every σ_j, and δ through `applied_residual_roundoff_cutoff`. A reported zero therefore means "not resolved above the retained binary64 cancellation cutoff," not a proof of exact mathematical or scientific equivalence.

2.7 Backend validation bounds

For a nonnegative scale x and positive operation-count bound k, define

$$\beta(k, x) = k\left(\tau^{\text{abs}}_{\text{validation}} + \tau^{\text{rel}}_{\text{validation}} x\right).$$

The solver checks, in maximum-entry norm,

$$\begin{aligned}
\|U^\dagger U - I_n\|_{\max} &\le \beta(n, 1), \\
\|V^\dagger V - I_n\|_{\max} &\le \beta(n, 1), \\
\|U\Sigma V^\dagger - C\|_{\max} &\le \beta(n, \|C\|_{\max}).
\end{aligned}$$

For a returned full-rank Q, it also checks right unitarity and agreement between $\|AQ - B\|_F^2$ and the reconstructed optimum. Validation tolerances verify the software/backend correlation; they do not alter the rank cutoff.

2.8 Using the scalar API

The dependency-free matrix representation is an exact tuple of exact tuples containing exact built-in complex values. Lists, NumPy arrays, real entries, ragged rows, empty matrices, and nonfinite values are rejected rather than coerced.

```
from projectkoios.optimization.alignment.procrustes import (
    ComplexUnitaryProcrustesDefinition,
    ComplexUnitaryProcrustesMethod,
    ComplexUnitaryProcrustesSolverFactory,
    ComplexUnitaryProcrustesStatus,
)

source = (
    (1.0 + 0.0j, 0.0 + 0.0j),
    (0.0 + 0.0j, 1.0 + 0.0j),
)
target = (
    (0.0 + 1.0j, 0.0 + 0.0j),
    (0.0 + 0.0j, 1.0 + 0.0j),
)

definition = ComplexUnitaryProcrustesDefinition(
    source_matrix=source,
    target_matrix=target,
)
solver = ComplexUnitaryProcrustesSolverFactory.create(
    ComplexUnitaryProcrustesMethod.NUMPY_DENSE
)
result = solver.solve(definition)

if result.status is ComplexUnitaryProcrustesStatus.UNIQUE:
    transformation = result.transformation
else:
    transformation = None
    # Inspect rank and residual evidence under consumer-owned policy.
```

At full rank, apply the returned tuple matrix on the right using the consumer's chosen matrix implementation. At deficient rank, do not manufacture a transformation from unexposed backend null-space vectors.

2.9 Result interpretation

The scalar result provides:

`cross_singular_values` the complete nonincreasing spectrum of C;

`numerical_rank` the strict-cutoff count r;

`applied_rank_cutoff` the actual τ_{rank};

`status` `UNIQUE` or `RANK_DEFICIENT`;

`transformation` Q only when status is `UNIQUE`;

`source_squared_frobenius_norm` p_A;

`target_squared_frobenius_norm` p_B;

`optimal_squared_residual` the roundoff-aware r_*^2;

`optimal_residual_frobenius_norm` $\sqrt{r_*^2}$;

`applied_residual_roundoff_cutoff` the squared-unit cutoff δ;

`backend_identifier` the executed API-level operation; and

`backend_version` the backend package version.

Backend metadata records the invoked API and package version, not linked-library provenance.

2.10 Failure handling

- `TypeError` reports malformed exact-type inputs.
- `ValueError` reports invalid values or inconsistent shapes.
- `FloatingPointError` reports nonfinite or overflowing arithmetic.
- `RuntimeError` reports NumPy SVD nonconvergence.

Numerical rank deficiency is not an exception: it is a successful status-bearing result.

Chapter 3

Batched complex-unitary Procrustes

3.1 Batched problem

Let $\mathcal{B} = (d_1, \ldots, d_k)$, with $k \geq 1$, be the leading batch shape. A source and target have shape

$$\mathcal{B} + (m, n), \qquad m > 0, \quad n > 0,$$

and equal complex entries at each leading multi-index $b \in \mathcal{B}$. For each b, the capability solves

$$\min_{Q_b^\dagger Q_b = I_n} \|A_b Q_b - B_b\|_F.$$

All leading dimensions and their C-order indexing are preserved. The capability neither flattens topology into a public result nor interprets adjacency between items.

3.2 Per-item mathematical procedure

The array implementation evaluates the scalar mathematics without creating one Python definition or result object per item. For every leading index b it performs the following operations:

$$\begin{aligned}
C_b &= A_b^\dagger B_b, \\
C_b &= U_b \Sigma_b V_b^\dagger, \\
\widetilde{Q}_b &= U_b V_b^\dagger, \\
\tau_b &= \tau_{\text{rank}}^{\text{abs}} + \tau_{\text{rank}}^{\text{rel}} \sigma_{b,1}, \\
r_b &= \sum_{j=1}^{n} \mathbf{1}[\sigma_{b,j} > \tau_b], \\
\rho_b^2 &= \|A_b\|_F^2 + \|B_b\|_F^2 - 2 \sum_{j=1}^{n} \sigma_{b,j}.
\end{aligned}$$

The last identity is evaluated with the scaled, roundoff-aware procedure in the scalar chapter. Thus each item also receives its own squared-unit cancellation cutoff δ_b.

The selected completion $\widetilde{Q}_b$ always acts on the right:

$$\widetilde{A}_b = A_b \widetilde{Q}_b.$$

If $r_b = n$, $\widetilde{Q}_b$ is the unique polar factor under the declared numerical rank policy. If $r_b < n$, it is only the completion selected by the backend's complete SVD and can change with an equally valid choice of null-space bases.

3.3 Array evidence and shapes

For input shape $\mathcal{B} + (m, n)$, the result has the following shapes:

`right_unitary_completions` $\mathcal{B} + (n, n)$, complex128;

`cross_singular_values` $\mathcal{B} + (n)$, float64;

`numerical_ranks` $\mathcal{B}$, integer;

`full_rank` $\mathcal{B}$, Boolean;

`applied_rank_cutoffs` $\mathcal{B}$, float64;

`source_squared_frobenius_norms` $\mathcal{B}$, float64;

`target_squared_frobenius_norms` $\mathcal{B}$, float64;

`optimal_squared_residuals` $\mathcal{B}$, float64;

`optimal_residual_frobenius_norms` $\mathcal{B}$, float64; and

`applied_residual_roundoff_cutoffs` $\mathcal{B}$, float64.

All arrays are immutable and relationally validated. Complete spectra, input powers, and roundoff cutoffs allow a consumer to reproduce each rank and residual decision.

The result also provides exact derived summaries:

- `batch_shape` is $\mathcal{B}$;
- `batch_size` is $\prod_i d_i$, including zero;
- `column_count` is n;
- `all_full_rank` is the Boolean reduction of `full_rank`;
- `rank_deficient_count` counts false entries; and
- `first_rank_deficient_index` is the first false index in deterministic C order, or `None`.

3.4 Fail-closed use of transformations

The field `right_unitary_completions` deliberately preserves homogeneous output, including diagnostic completions. A consumer that requires uniquely identified transformations must use

```
transformations = result.require_unique_right_unitary_transformations()
```

This returns the existing immutable completion array, without copying, only if every item is full rank. Otherwise it raises `ValueError` and identifies the first deficient leading index in C order.

If a consumer intentionally inspects deficient completions, it must pair them with `full_rank`, `numerical_ranks`, and `applied_rank_cutoffs` and must not report them as unique transformations. The library does not decide whether one deficient item invalidates an entire scientific path, plaquette, campaign, or other topology.

3.5 Empty batches

At least one leading batch dimension is required, but a leading dimension may be zero. For example, shape $(4, 0, m, n)$ is valid. An empty batch:

- preserves every exact output shape;
- invokes no SVD backend;
- carries `None` for backend identifier and version;
- has `rank_deficient_count==0`;
- has `first_rank_deficient_indexisNone`; and
- is vacuously `all_full_rank`.

The unique-transformation accessor consequently returns the shape-preserving empty completion array. None of these vacuous properties claims that a transformation was computed.

3.6 Whole-batch failure policy

Input validation occurs before backend execution. Backend evidence and all numerical relations are validated per item. The first numerical failure is reported in deterministic C order when an index is available. A failure rejects the whole batch; there is no partial result. Numerical rank deficiency remains a successful result and is not treated as backend failure.

3.7 Memory model

For batch size $N = \prod_i d_i$, cross matrices, SVD factors, and completions each scale as $O(Nn^2)$; complete spectra scale as $O(Nn)$; and scalar evidence scales as $O(N)$. The implementation does not create nested Python tuple matrices for individual items. Consumers should account for these required arrays when choosing a batch size.

Chapter 4

NumPy batched Procrustes

Supported façade: `projectkoios.optimization.alignment.numpy.batched_procrustes`

4.1 Input contract

This backend realizes the itemwise method with NumPy's dense array and linear-algebra operations [7]. The source and target must be exact `numpy.ndarray` objects with exact `numpy.complex128` dtype and common shape `(...,m,n)`. There must be at least one leading batch dimension; m and n must be positive. Zero-sized leading dimensions are allowed. Every real and imaginary component must be finite.

The API deliberately rejects ndarray subclasses, complex64, real, object, Boolean, and implicitly coercible inputs. If consumer data starts in another representation, convert it explicitly before creating the definition:

```
import numpy as np

source = np.asarray(source_data, dtype=np.complex128)
target = np.asarray(target_data, dtype=np.complex128)
```

This conversion is consumer-visible and separate from contract validation.

4.2 Normal construction: defensive copies

Use `from_arrays(...)` for ordinary inputs. It validates exact array type, exact dtype, and finiteness, then stores independent C-order arrays backed by immutable bytes. Later changes to the caller's inputs cannot change the definition, and NumPy writeability cannot be re-enabled on the stored arrays.

```
from projectkoios.optimization.alignment.numpy.batched_procrustes import (
    NumpyBatchedComplexUnitaryProcrustesDefinition,
    NumpyBatchedComplexUnitaryProcrustesSolver,
)
from projectkoios.optimization.alignment.procrustes import (
    ComplexUnitaryProcrustesTolerance,
)

tolerance = ComplexUnitaryProcrustesTolerance(
    rank_absolute=0.0,
```

```
    rank_relative=1.0e-12,
    validation_absolute=1.0e-12,
    validation_relative=1.0e-10,
)

definition = NumpyBatchedComplexUnitaryProcrustesDefinition.from_arrays(
    source_matrices=source,
    target_matrices=target,
    tolerance=tolerance,
)
result = NumpyBatchedComplexUnitaryProcrustesSolver().solve(definition)
```

The definition exposes `batch_shape`, `batch_size`, `row_count`, and `column_count` derived from its immutable arrays.

4.3 Solving and applying the transformation

For a nonempty batch, the solver forms all $A_b^\dagger B_b$ cross matrices and makes one public batched `numpy.linalg.svd` call. It selects `left@right_adjoint`, validates every item, and constructs an immutable result.

A consumer requiring unique transformations should first use the fail-closed accessor and then apply the returned array on the right:

```
transformations = result.require_unique_right_unitary_transformations()
aligned_source = definition.source_matrices @ transformations

# Per-item objective values and resolution limits:
residual_norm = result.optimal_residual_frobenius_norms
residual_squared = result.optimal_squared_residuals
residual_resolution = result.applied_residual_roundoff_cutoffs
```

For multidimensional leading shapes, NumPy matrix multiplication preserves those leading dimensions. The aligned result has the same shape as the source.

If diagnostic deficient completions are needed, access them explicitly and preserve the rank mask alongside them:

```
diagnostic_completions = result.right_unitary_completions
unique_mask = result.full_rank
first_deficient = result.first_rank_deficient_index
```

Do not treat entries where `unique_mask` is false as uniquely identified transformations.

4.4 Complete worked example

The following example constructs a two-item batch, solves it, checks uniqueness, applies the transformations, and verifies the public residual evidence.

```
import numpy as np

from projectkoios.optimization.alignment.numpy.batched_procrustes import (
    NumpyBatchedComplexUnitaryProcrustesDefinition,
    NumpyBatchedComplexUnitaryProcrustesSolver,
)
```

```
source = np.asarray(
    [
        [[1.0 + 0.0j, 0.0 + 0.0j],
         [0.0 + 0.0j, 1.0 + 0.0j]],
        [[1.0 + 0.0j, 0.0 + 0.0j],
         [0.0 + 0.0j, 2.0 + 0.0j]],
    ],
    dtype=np.complex128,
)
known = np.asarray(
    [
        [[0.0 + 1.0j, 0.0 + 0.0j],
         [0.0 + 0.0j, 1.0 + 0.0j]],
        [[0.0 + 0.0j, 1.0 + 0.0j],
         [1.0 + 0.0j, 0.0 + 0.0j]],
    ],
    dtype=np.complex128,
)
target = source @ known

definition = NumpyBatchedComplexUnitaryProcrustesDefinition.from_arrays(
    source_matrices=source,
    target_matrices=target,
)
result = NumpyBatchedComplexUnitaryProcrustesSolver().solve(definition)
transformations = result.require_unique_right_unitary_transformations()
aligned = definition.source_matrices @ transformations

np.testing.assert_allclose(aligned, definition.target_matrices)
np.testing.assert_allclose(
    np.sum(np.abs(aligned - definition.target_matrices) ** 2, axis=(-2, -1)),
    result.optimal_squared_residuals,
    atol=1.0e-12,
    rtol=1.0e-10,
)
```

The assertion tolerances in an application remain consumer choices. The result itself has already been checked using its declared validation policy.

4.5 Zero-copy immutable construction

Advanced callers may use `from_immutable_arrays(...)` to retain arrays by identity. It performs validation but no copying. Each supplied array must already satisfy all of the following conditions:

- exact `numpy.ndarray` type;
- exact required dtype;
- finite entries;
- C-contiguous layout;
- read-only status; and

- a NumPy base chain terminating in an exact built-in `bytes` object.

A read-only view over mutable storage is not sufficient, and merely calling `setflags(write=False)` does not establish immutable ownership. Non-C-contiguous immutable views are also rejected.

The public numerical helper can create the canonical representation, although that conversion itself necessarily copies ordinary storage:

```
import numpy as np
from projectkoios.numerics.numpy.validation import immutable_array
from projectkoios.optimization.alignment.numpy.batched_procrustes import (
    NumpyBatchedComplexUnitaryProcrustesDefinition,
)

immutable_source = immutable_array(
    source, "source_matrices", np.dtype(np.complex128)
)
immutable_target = immutable_array(
    target, "target_matrices", np.dtype(np.complex128)
)
definition = (
    NumpyBatchedComplexUnitaryProcrustesDefinition.from_immutable_arrays(
        source_matrices=immutable_source,
        target_matrices=immutable_target,
    )
)
assert definition.source_matrices is immutable_source
assert definition.target_matrices is immutable_target
```

Zero-copy construction is useful when a caller already owns canonical arrays or passes arrays between immutable owner records. It does not provide a shortcut for adopting mutable caller storage.

The raw frozen dataclass constructor has the same canonical immutable-array requirement. Use one of the named factories so copying intent is explicit.

4.6 Result construction and persistence

Ordinary users receive results from the solver. Code that reconstructs a result from trusted persisted evidence may use `NumpyBatchedComplexUnitaryProcrustesResult.from_arrays(...)`. That factory makes independent immutable copies and then validates all shapes and mathematical relations. Its required exact array dtypes are:

- complex128 for `right_unitary_completions`;
- float64 for singular values, cutoffs, powers, and residual evidence;
- `numpy.intp` for `numerical_ranks`; and
- `numpy.bool_` for `full_rank`.

The keyword-only scalar fields are positive exact built-in `row_count`, the exact owner tolerance record, and coupled backend identifier/version strings. Both backend strings must be present and nonempty for a nonempty batch; both must be `None` for an empty batch.

Use `from_immutable_arrays(...)` only when every result array already satisfies the canonical immutable-storage conditions above. It retains all arrays by identity and still performs complete

relational validation. Construction is not a way to assert unverified evidence: inconsistent spectra, ranks, cutoffs, residuals, completion unitarity, shapes, or backend metadata are rejected.

When persisting a result, retain at least the complete result fields and tolerance record. Persisting only residual norms loses the declared rank decision and the roundoff resolution needed to interpret a zero residual.

4.7 Backend validation performed before return

For every nonempty item, the NumPy solver checks:

- finite, nonnegative, nonincreasing complete singular values;
- left and right singular-vector unitarity;
- reconstruction of $A_b^\dagger B_b$;
- right unitarity of the selected completion;
- finite source and target powers; and
- agreement of the direct squared residual $\|A_b\widetilde{Q}_b - B_b\|_F^2$ with the reconstructed optimum.

The dimension-scaled bound $\beta(k, x)$ defined in the scalar chapter uses $k = n$ for unitary and cross-reconstruction checks. Direct residual agreement uses $k = mn$ and the scale

$$\max\left(\|A_b\|_F^2, \|B_b\|_F^2, \rho_b^2\right).$$

4.8 Errors and recovery

`TypeError` The definition/result type, ndarray type, dtype, tolerance type, or another nominal type is wrong.

`ValueError` Shapes, finiteness at construction, immutable-storage requirements, relational evidence, or a validation bound is invalid. The unique accessor also uses this error for rank deficiency.

`FloatingPointError` Cross products, powers, residual reconstruction, cutoffs, or backend output become nonfinite, or a reconstructed residual is materially negative.

`RuntimeError` NumPy SVD does not converge.

A rare failed batched SVD may trigger scalar diagnostic retries solely to identify the first reproducibly failing C-order batch index. No partial successful batch is returned. The appropriate recovery is to correct the representation, revise an explicitly owned software tolerance when justified, split or rescale consumer data if appropriate, or record the failure under consumer policy. Suppressing rank evidence or treating a failed batch as partially valid is not supported.

Chapter 5

SciPy batched Procrustes status

No SciPy batched Procrustes definition, solver, result, or façade is currently implemented or exported. Installing the `scipy` optional dependency does not add a batched Procrustes API.

For dense exact complex128 batches, use the NumPy procedure in the preceding chapter:

```
from projectkoios.optimization.alignment.numpy.batched_procrustes import (
    NumpyBatchedComplexUnitaryProcrustesDefinition,
    NumpyBatchedComplexUnitaryProcrustesSolver,
)
```

Do not infer SciPy Procrustes availability from unrelated SciPy capabilities in the distribution, and do not import an internal or guessed SciPy alignment path.

A future SciPy backend would require a distinct representation or execution need, an explicit public contract, independent numerical qualification, and isolated-wheel verification. It will not be introduced as a name-only wrapper around NumPy.

References

[1] P. H. Schönemann, "A generalized solution of the orthogonal Procrustes problem," *Psychometrika*, vol. 31, pp. 1–10, 1966. doi:10.1007/BF02289451.

[2] J. C. Gower and G. B. Dijksterhuis, *Procrustes Problems*. Oxford University Press, Oxford Statistical Science Series 30, 2004. ISBN 978-0-19-851058-1.

[3] N. J. Higham, "Computing the polar decomposition—with applications," *SIAM Journal on Scientific and Statistical Computing*, vol. 7, no. 4, pp. 1160–1174, 1986. doi:10.1137/0907079.

[4] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013. ISBN 978-1-4214-0794-4.

[5] R. Bhatia, *Matrix Analysis*. Springer, Graduate Texts in Mathematics 169, 1997. doi:10.1007/978-1-4612-0653-8.

[6] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Society for Industrial and Applied Mathematics, 2002. doi:10.1137/1.9780898718027.

[7] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. doi:10.1038/s41586-020-2649-2.

Verification scope

Software verification and bounded numerical verification establish implementation properties only. They do not establish scientific validity, material-model adequacy, or human acceptance.